

RVT-Swarm: Recoverability-Verified Topological Control for Decentralized Swarm Formation

Anonymous Authors

Abstract—Decentralized swarm formation in cluttered environments requires more than instantaneous collision avoidance: the team must also preserve the ability to reorganize and recover a usable formation after passing through bottlenecks. We present RVT-Swarm, a graph-based controller that augments per-robot acceleration commands with discrete topology actions and a learned *recoverability* score map over candidate formations. Recoverability is defined as a short-horizon ability to remain collision-free, make goal progress, and return toward a formation tube under a candidate topology. RVT-Swarm combines (i) a local interaction-graph encoder with action, recoverability, uncertainty, and topology heads; (ii) a counterfactual topology selector with hysteresis and cooldown; and (iii) a lightweight CBF-inspired progress-preserving projection shield. Training uses imitation learning plus counterfactual short-horizon rollouts from a common expert controller. On the current prototype benchmark (4 scenarios, 4–16 robots), RVT-Swarm reaches the best composite success rate among all tested methods at 43.5%, compared with 41.25% for InstantCert and 39.0% for GNN-Only, while also reducing deadlock to 6.75% and irreversible collapse to 11.25%. The ablation study reveals a more nuanced picture: the full method improves collision-free execution and overall benchmark ranking, but some component removals improve individual metrics, indicating that the current recoverability surrogate and end-task objective are not yet perfectly aligned. We therefore frame RVT-Swarm as a strong research scaffold for recoverability-aware swarm formation control rather than a final benchmark endpoint.

Index Terms—Multi-robot systems, swarm formation control, graph neural networks, safety-critical control, topology adaptation, recoverability.

I. INTRODUCTION

Multi-robot formations are useful only when they remain both *safe* and *task-capable*. In warehouse logistics, cooperative inspection, or search-and-rescue, a team must not merely avoid collisions; it must also preserve enough structure to pass through clutter, regroup, and continue making progress toward a collective goal [1], [2], [3]. This is especially difficult in decentralized settings with limited local sensing, heterogeneous local contexts, and team sizes that vary at deployment time.

Existing approaches usually solve only part of this problem. Graph-based decentralized controllers scale well to large teams but often optimize local motion directly, without an explicit notion of whether the team can still *recover* a useful formation after local reconfiguration [4], [5]. Control-barrier-function (CBF) approaches provide strong instantaneous safety structure [6], [7], [8], but they typically reason about collision

avoidance rather than return to a formation tube. Adaptive-formation approaches can change shape to fit the environment, but usually rely on hand-designed switching logic or implicit policies without an explicit recoverability score over alternative topologies [2], [3]. Reactive methods such as ORCA can maintain goal progress, but often do so by sacrificing formation quality entirely [9].

This paper explores a different angle: rather than asking only whether the current action is safe, we ask whether the current local state is *recoverable* under a small set of candidate formation topologies. In RVT-Swarm, each local graph observation is mapped not only to a control action but also to a score map over topological actions (KEEP, COMPRESS, LINE, SPLIT, RECOVER). The controller then evaluates these topologies counterfactually and selects the one with the best predicted recoverability, subject to hysteresis and cooldown. A lightweight projection-based shield intervenes only when risk is high or when all candidate topologies appear unrecoverable.

The resulting system is deliberately pragmatic. It does not claim an exact viability kernel or a formal distributed reachability proof. Instead, it uses short-horizon counterfactual rollouts to construct a learned surrogate of recoverability and then deploys that surrogate online for topology adaptation. This design is consistent with the current codebase and the current benchmark setting.

A. Contributions

We make the following contributions:

- 1) We formulate *recoverability-aware topology adaptation* for decentralized swarm formation, where candidate formation modes are scored by their short-horizon ability to remain safe, make goal progress, and return toward a formation tube.
- 2) We implement a graph-attention controller with four coupled outputs: per-robot accelerations, graph-level topology logits, per-topology recoverability scores, and uncertainty estimates. A gradient-scaling mechanism reduces interference from auxiliary heads on the action pathway.
- 3) We introduce a recoverability-aware topology selector with hysteresis, cooldown, contextual bonuses, and a progress-preserving CBF-inspired projection shield that only activates under elevated risk or global negative recoverability.
- 4) We provide a unified experimental scaffold with learned baselines, historical baselines, rollout-based labels, and

liveness metrics such as deadlock, topology switching, and irreversible collapse. On the current prototype benchmark, RVT-Swarm attains the best overall success among tested methods.

II. RELATED WORK

A. Decentralized Graph Policies

Graph neural networks are a natural fit for decentralized multi-robot control because they preserve permutation invariance and local communication structure [4], [5]. Our work follows this line in using a local interaction graph, but adds explicit topology adaptation and a recoverability score map rather than predicting control alone.

B. Safety-Critical Multi-Robot Control

Recent safety-critical multi-robot controllers use graph-structured certificates, collaborative safety filters, or discrete safe-policy learning to enforce pairwise constraints in distributed teams [6], [7], [8]. Recent graph-CBF work scales safety reasoning through local graphs [6], [8]. RVT-Swarm is influenced by this design, but our shield is intentionally lightweight and approximate: it is a projection-based correction mechanism rather than a formally certified QP solver. More importantly, our main decision variable is not only the continuous action but also the topology under which the swarm remains recoverable.

C. Adaptive Formation and Recoverability

Adaptive formation methods change the team geometry to fit corridors, obstacles, or task context [2], [3]. Viability and reachability learning study whether a state can remain in or return to a desirable set over a finite horizon [10], [11], [12]. RVT-Swarm sits between these directions: it learns a short-horizon, local-graph surrogate of recoverability and uses it to guide topology changes. The current implementation should therefore be understood as a recoverability-informed formation controller rather than as a formal reachability engine.

To make this positioning explicit, Table I contrasts RVT-Swarm with representative prior work along the axes that matter most for the current prototype: local graph execution, formation awareness, explicit topology adaptation, recoverability modeling, counterfactual selection, and progress-preserving safety intervention.

III. PROBLEM SETUP

A. Environment and Dynamics

We consider N robots moving in a 2D workspace with discrete-time kinematics

$$\mathbf{p}_i(t+1) = \mathbf{p}_i(t) + \mathbf{v}_i(t)\Delta t, \quad \mathbf{v}_i(t+1) = \mathbf{v}_i(t) + \mathbf{u}_i(t)\Delta t, \quad (1)$$

subject to bounded speed and acceleration. Each robot observes local relative geometry, obstacle context, goal direction, formation error, and a 36-ray 270° LiDAR scan. The environment includes static obstacles, dynamic obstacles, and a narrow-passage configuration.

B. Formation Tube and Topology Actions

Let $\bar{\mathbf{p}}(t)$ denote the swarm centroid and let $\{\delta_i^\tau\}_{i=1}^N$ denote the desired offsets of topology $\tau \in \mathcal{T}$. The current formation error is

$$\mathbf{e}_i^\tau(t) = \bar{\mathbf{p}}(t) + \delta_i^\tau - \mathbf{p}_i(t). \quad (2)$$

Rather than choosing only a continuous control input, the policy also selects a discrete *formation mode* (or topology action) from a small finite set:

$$\mathcal{T} = \{\text{KEEP, COMPRESS, LINE, SPLIT, RECOVER}\}. \quad (3)$$

Intuitively, these modes tell the swarm whether to keep its current shape, compress into a tighter pattern, align into a line, temporarily split into subgroups, or recover back toward the nominal formation. These actions control the global formation geometry through scale, corridor alignment, or temporary splitting/merging.

C. Recoverability

For a state s_t and topology τ , we define recoverability as a short-horizon score that rewards collision-free motion, goal progress, and return toward a formation tube while penalizing deadlock, excessive switching, and persistent split states:

$$R(s_t; \tau) = \frac{1}{H} \sum_{h=0}^{H-1} \left[w_c c_{t+h} + w_p p_{t+h} + w_f f_{t+h} - w_d d_{t+h} - w_s q_{t+h} \right]. \quad (4)$$

Here c_{t+h} is a collision-free indicator, p_{t+h} measures goal progress, f_{t+h} measures formation-tube recovery, d_{t+h} is a deadlock-related penalty, and q_{t+h} captures switching/splitting penalties. In the implementation, these scores are generated by short-horizon counterfactual rollouts of a common expert controller.

The online objective is then to select $(\mathbf{u}, \tau)$ that preserves progress while favoring positive recoverability:

$$\max_{\mathbf{u}, \tau} \alpha \text{progress} + \beta \text{formation} + \gamma \hat{R}(s_t; \tau), \quad (5)$$

where $\hat{R}$ is the learned recoverability surrogate.

IV. METHOD

RVT-Swarm combines a graph-attention backbone, a recoverability-aware multi-head decoder, a counterfactual topology selector, and a projection-based shield. Figure 1 shows the overall pipeline and Figure 2 details the recoverability network.

TABLE I: Positioning of RVT-Swarm relative to representative recent prior work. ✓ = primary capability; △ = partially addressed; ✗ = not addressed. The table is intended as a qualitative positioning summary for the current prototype rather than an exhaustive taxonomy.

Method	Venue	Year	Local graph distributed	Formation aware	Obstacle dynamics	Topology adaptation	Recoverability model	Counterfactual selection	Progress shield
GCBF+ [6]	T-RO	2025	✓	△	✓	✗	✗	✗	△
Collab. safety form. [7]	arXiv	2024	✓	✓	△	✗	✗	✗	△
Neural barrier reach [10]	IFAC	2024	✗	✗	△	✗	✓	✗	✗
Cert. Reach. [11]	RA-L	2025	△	✗	✓	✗	✓	✗	✗
RAIL [12]	ICRA	2025	△	✗	✓	✗	✓	✗	✗
DGPPO [8]	ICLR	2025	✓	✗	✓	✗	✗	✗	✓
Form.-adapt nav. [2]	arXiv	2024	✓	✓	△	✓	✗	✗	✗
Multi-UAV RL [3]	IROS	2025	✓	✓	✓	△	✗	✗	✗
RVT-Swarm (ours)	—	2026	✓	✓	✓	✓	✓	✓	✓

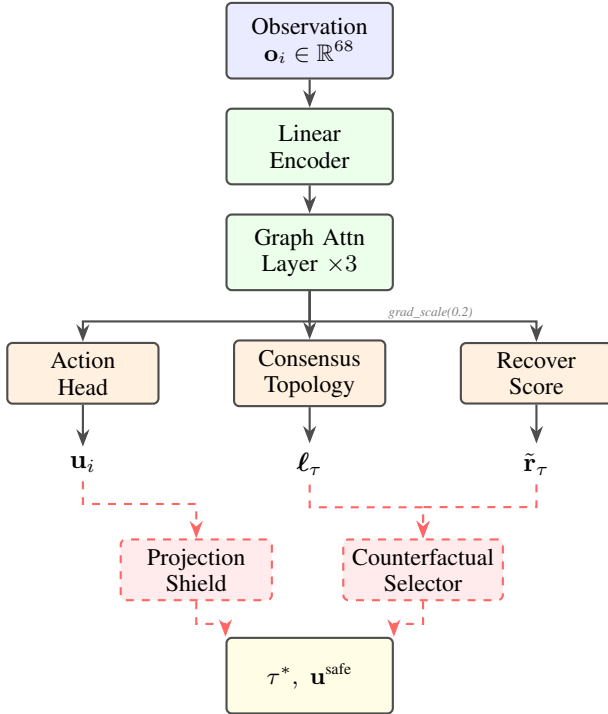


Fig. 1: Overview of RVT-Swarm. Local graph observations feed a shared graph-attention encoder whose outputs are decoded into control actions, topology logits, and per-topology recoverability scores. A counterfactual selector and a lightweight projection shield operate only at inference time.

A. Local Graph Encoder

At each step, the swarm builds a k -nearest-neighbor interaction graph with $k = 6$. Each node contains a 68-dimensional feature vector including position, velocity, heading, goal vector, centroid-relative position, formation error, obstacle-relative cues, topology encoding, normalized minimum TTC proxy, and 36 normalized LiDAR ranges. Each edge contains 11-dimensional relational features, including relative position, relative velocity, spacing error, corridor projections, and a TTC-style safety descriptor.

The graph is processed by three attention-weighted message-passing layers. For edge $(j \rightarrow i)$, an edge MLP

produces a message $\mathbf{m}_{ij}$ and a learned attention weight α_{ij} , and the node update is

$$\mathbf{m}_{ij} = \phi_e([\mathbf{h}_i \parallel \mathbf{h}_j \parallel \mathbf{e}_{ij}]), \quad (6)$$

$$\alpha_{ij} = \text{softmax}_{j \in \mathcal{N}(i)}(\phi_a(\mathbf{m}_{ij})), \quad (7)$$

$$\mathbf{h}'_i = \mathbf{h}_i + \phi_n\left([\mathbf{h}_i \parallel \sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{m}_{ij}]\right). \quad (8)$$

The hidden dimension is 128 throughout.

B. Recoverability Field Network

The shared backbone feeds four decoders:

- an **action head** producing per-robot accelerations $\mathbf{u}_i \in \mathbb{R}^2$;
- a **topology consensus head** producing graph-level logits over $\mathcal{T}$;
- a **score head** producing per-topology recoverability scores;
- an **uncertainty head** producing a nonnegative uncertainty estimate for each candidate topology.

The topology head is not a simple pooled classifier. Each node first casts a topology vote, neighbor votes are aggregated into mean and spread statistics, and a small agreement MLP refines these votes before graph-level pooling. This gives the model a limited form of local consensus without requiring explicit communication optimization. The consensus path is summarized in Algorithm 1.

Algorithm 1: Neighborhood Topology Consensus

Input: Node embeddings $\{\mathbf{h}_i\}$ and graph neighborhood $\mathcal{N}(i)$

Output: Graph-level topology logits ℓ

- 1 $\mathbf{v}_i \leftarrow \text{MLP}_{\text{vote}}(\mathbf{h}_i) \quad \forall i$;
- 2 $\bar{\mathbf{v}}_i \leftarrow \text{MeanAgg}_{j \in \mathcal{N}(i)}(\mathbf{v}_j)$;
- 3 $\sigma_i \leftarrow \sqrt{\text{VarAgg}_{j \in \mathcal{N}(i)}(\mathbf{v}_j) + \epsilon}$;
- 4 $\hat{\mathbf{v}}_i \leftarrow \text{MLP}_{\text{agree}}([\mathbf{v}_i \parallel \bar{\mathbf{v}}_i \parallel \sigma_i])$;
- 5 $\ell \leftarrow \text{MeanPool}(\hat{\mathbf{v}}_i)$;
- 6 **return** ℓ

Recoverability scores are adjusted by uncertainty:

$$\tilde{r}_\tau = \hat{r}_\tau - 0.25 \hat{\sigma}_\tau, \quad \hat{R}(s_t) = \max_{\tau \in \mathcal{T}} \tilde{r}_\tau. \quad (9)$$

Only the action head receives full-strength gradients from the backbone. Before feeding the auxiliary heads, we apply

gradient scaling with factor 0.2 to reduce degradation of control quality under multitask training.

C. Counterfactual Topology Selection

At inference, RVT-Swarm evaluates all candidate topologies using a combination of recoverability score, topology prior, contextual bonuses, uncertainty penalty, and switching cost:

$$C_\tau = 0.62 \tilde{r}_\tau + 0.22 \pi_\tau + b_\tau^{\text{ctx}} - 0.08 \sigma_\tau - \psi_\tau^{\text{switch}}. \quad (10)$$

The context term favors, for example, LINE or COMPRESS in bottlenecks and RECOVER once the bottleneck clears. The switch penalty discourages excessive topology changes, especially disruptive split actions. A new topology is adopted only if it exceeds the previous topology by a hysteresis margin, and a cooldown period suppresses immediate repeated switching. The online selection-and-shield loop is summarized in Algorithm 2.

Algorithm 2: Counterfactual Topology Selection with Projection Shield

Input: Observation o_t , topology logits ℓ , adjusted scores $\tilde{r}$, uncertainty σ , previous topology τ_{prev} , nominal action $\mathbf{u}$
Output: Selected topology τ^* , shielded action $\mathbf{u}^{\text{safe}}$

```

1 if  $t - t_{\text{last-switch}} < T_{\text{cool}}$  then
2   |  $\tau^* \leftarrow \tau_{\text{prev}}$ ;
3 end
4 else
5   | Compute prior  $\pi = \text{softmax}(\ell/T)$ ;
6   | Compute contextual bonuses  $b_\tau^{\text{ctx}}$ ;
7   | Compute switching penalties  $\psi_\tau^{\text{switch}}$ ;
8   | Compute  $C_\tau$  using (10);
9   |  $\tau' \leftarrow \arg \max_{\tau \in \mathcal{T}} C_\tau$ ;
10  | if  $C_{\tau'} < C_{\tau_{\text{prev}}} + \eta$  then
11    |  $\tau^* \leftarrow \tau_{\text{prev}}$ ;
12  | else
13    |  $\tau^* \leftarrow \tau'$ ;
14  | end
15 end
    // Risk-triggered projection shield
16  $\rho \leftarrow \text{CollisionRisk}(o_t)$ ;
17 if  $\max_{\tau \in \mathcal{T}} \tilde{r}_\tau < 0$  then
18   |  $\rho \leftarrow \max(\rho, \rho_{\text{crisis}})$ ;
19 end
20 if  $\rho \geq \rho_{\text{th}}$  then
21   | Build pairwise robot-robot and robot-obstacle half-planes;
22   | Project  $\mathbf{u}$  onto the feasible intersection and acceleration ball;
23   | Blend projected and nominal actions to obtain  $\mathbf{u}^{\text{safe}}$ ;
24 end
25 else
26   |  $\mathbf{u}^{\text{safe}} \leftarrow \mathbf{u}$ ;
27 end
28 return  $\tau^*, \mathbf{u}^{\text{safe}}$ 

```

D. Progress-Preserving Projection Shield

The shield activates only when collision risk is high or when all candidate topologies have negative recoverability. Pairwise robot-robot and robot-obstacle constraints are converted into half-plane constraints in action space using a CBF-inspired linearization. For robot i , we use the barrier

$$h_{ij} = \|\mathbf{p}_i - \mathbf{p}_j\|^2 - d_{\text{safe}}^2 \quad (11)$$

and enforce

$$\nabla h_{ij}^\top \mathbf{u}_i \geq -\alpha h_{ij} - 2(\mathbf{p}_i - \mathbf{p}_j)^\top (\mathbf{v}_i - \mathbf{v}_j). \quad (12)$$

Rather than solving a full external QP, we iteratively project a progress-biased target action onto the intersection of these half-planes and the acceleration ball. The final shielded action is softly blended with the learned action, which reduces overreaction near the risk threshold.

V. TRAINING

A. Data Generation

Training data are generated from a common heuristic expert controller in the shared simulator. For each sampled state, we run short-horizon counterfactual rollouts for every topology in $\mathcal{T}$ using the same expert controller, producing:

- a per-topology score vector;
- a best-topology label;
- a scalar recoverability margin derived from the score gap and the best score.

The current release uses 500 episodes spanning four scenarios (*open field*, *cluttered*, *narrow passage*, *dynamic obstacles*) and team sizes $N \in \{4, 8, 12, 16\}$. Hard-negative mining augments difficult bottleneck states with perturbed actions.

B. Loss and Optimization

For RVT-Swarm, the training loss is

$$\mathcal{L} = \lambda_a \mathcal{L}_{\text{act}} + \lambda_r \mathcal{L}_{\text{rec}} + \lambda_\tau (\mathcal{L}_{\text{topo}} + 0.8 \mathcal{L}_{\text{score}} + 0.45 \mathcal{L}_{\text{rank}}) + \lambda_x \mathcal{L}_{\text{aux}} + \mathcal{L}_{\text{unc}}, \quad (13)$$

where the terms correspond respectively to action regression, recoverability regression, topology classification, score-map regression, pairwise topology ranking, auxiliary environment-state prediction, and uncertainty regularization. The current hyperparameters are $\lambda_a = 5.0$, $\lambda_r = 0.02$, $\lambda_\tau = 0.01$, and $\lambda_x = 0.004$.

Auxiliary losses are linearly ramped in over a 50-epoch warmup for RVT-Swarm. The three learned methods are trained with AdamW (3×10^{-4} learning rate, 10^{-5} weight decay, batch size 32). Best-checkpoint selection is based on periodic rollout validation: after warmup, every five epochs we evaluate a small validation set over *narrow passage* and *dynamic obstacles* for team sizes 8 and 16, and select checkpoints using a score that combines success, collision-free execution, formation quality, deadlock, stall, and collapse.

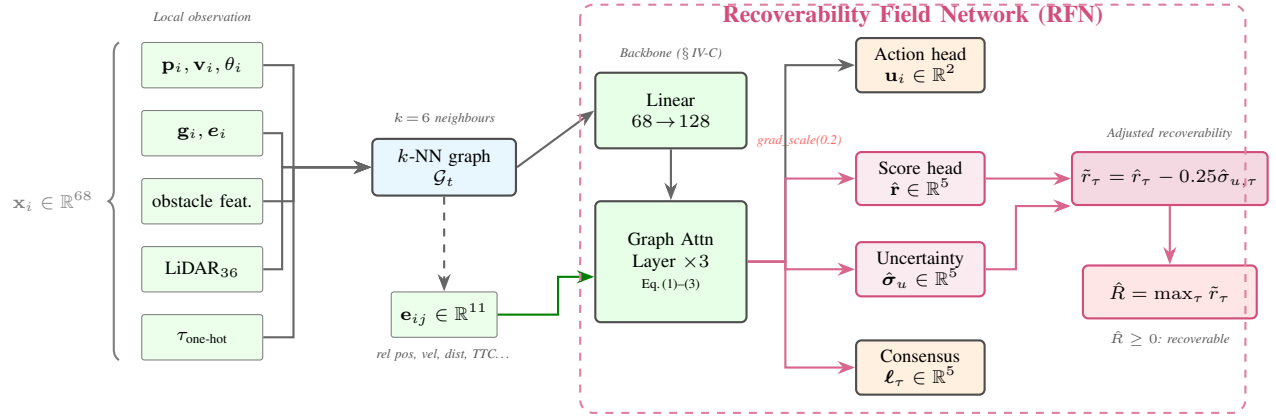


Fig. 2: Recoverability Field Network (RFN). Local node and edge features are encoded by a three-layer graph-attention backbone and decoded into action, recoverability, uncertainty, and topology outputs. The adjusted score map $\tilde{r}_\tau$ is used by the topology selector and by the shield trigger.

VI. EXPERIMENTS

A. Experimental Protocol

The benchmark uses a common 2D simulator with workspace 12×12 m, time step $\Delta t = 0.15$ s, robot radius 0.18 m, obstacle radius 0.35 m, speed limit 0.9 m/s, acceleration limit 0.6 m/s^2 , and a 36-ray LiDAR with 3 m range. Main evaluation uses 25 episodes per setting over 4 scenarios and 4 team sizes, for 400 episodes per method. Episodes are matched across methods by shared seeds.

a) *Implementation scope.*: This benchmark is intentionally a *common-simulator prototype*. The non-learned baselines (ADAPTIVE_FORMATION, CBF_QP_LIKE, ORCA_LIKE, and CENTRALIZED_MPC) are internal normalized baselines rather than exact package-level reproductions. The current paper reports the single-seed result produced by the released code path; multi-seed confidence intervals remain future work.

B. Compared Methods

GNN-Only uses the shared graph backbone with only an action head. **InstantCert** adds a scalar recoverability-style head but no topology adaptation. **RVT-Swarm** is the full model. Historical baselines include a heuristic adaptive-formation controller, a CBF-QP-like reactive filter, ORCA-like avoidance, and a centralized MPC-inspired proxy.

C. Main Results

Three points stand out from Table II. First, RVT-Swarm achieves the highest composite success among all methods. The margin over InstantCert is modest in absolute terms (+2.25 percentage points) but consistent with the intended role of recoverability-aware topology adaptation. Second, RVT-Swarm produces the highest formation-success rate among learned methods (**47.75%** `form_ok`) and the lowest deadlock rate among methods that still preserve formation. Third, the table exposes the trade-offs of the baselines: ORCA-like motion reaches the goal often but abandons formation entirely, while CBF-QP-like control maximizes collision-free execution at the cost of formation completion.

D. Ablation Results

TABLE III: Ablation results for RVT-Swarm. The current prototype ablation study is intentionally reported *as is*: the full model is not uniformly best on every metric.

Variant	Success $\uparrow$	Coll-Free $\uparrow$	Form OK $\uparrow$	Deadlock $\downarrow$	Collapse $\downarrow$	Topo Sw. $\downarrow$
Full	0.398	0.663	0.425	0.060	0.118	0.262
– Recoverability	0.405	0.640	0.438	0.090	0.140	0.455
– Counterfactual	0.405	0.638	0.450	0.045	0.095	0.820
– Progress shield	0.405	0.630	0.458	0.065	0.100	0.333
– Topology	0.400	0.625	0.438	0.095	0.133	0.000
Fixed formation	0.385	0.655	0.415	0.080	0.140	0.000

The ablation study is the most important reason to keep the current paper honest. The full model is *best on the main benchmark*, but it is not best on every ablation metric. Removing recoverability, counterfactual selection, or the progress shield slightly improves success in this particular ablation sweep, whereas the full model achieves the best collision-free rate. Meanwhile, removing counterfactual scoring produces very high topology switching (0.82), suggesting that the counterfactual selector is stabilizing the policy even when its effect on success is not yet dominant. Taken together, these results support a nuanced conclusion: the RVT ingredients matter, but their present coupling is still imperfect.

E. Interpretation

The results suggest that RVT-Swarm wins primarily by balancing formation completion against liveness failure modes. Compared with InstantCert and GNN-Only, it reduces deadlock and collapse while keeping collision-free performance competitive. At the same time, the ablation table shows that the recoverability proxy and the end-task objective are not yet perfectly aligned. This is consistent with the codebase design: recoverability labels are short-horizon counterfactual surrogates rather than exact viability certificates, and the selector/shield operates with simple contextual bonuses rather than a fully optimized planner.

TABLE II: Main benchmark results averaged over all scenarios and team sizes. Numbers are from the current single-seed prototype run.

Method	Success $\uparrow$	Goal $\uparrow$	Coll-Free $\uparrow$	Form OK $\uparrow$	Form RMS $\downarrow$	Stall $\downarrow$	Deadlock $\downarrow$	Collapse $\downarrow$	ms/step $\downarrow$
ORCA-like	0.000	0.932	0.625	0.000	2.930	0.000	0.000	0.020	1.0
Adaptive Formation	0.288	0.785	0.565	0.348	0.952	0.026	0.075	0.120	1.0
CBF-QP-like	0.188	0.728	0.718	0.210	0.842	0.028	0.093	0.095	1.0
Centralized MPC	0.315	0.770	0.630	0.340	0.951	0.025	0.108	0.160	1.0
GNN-Only	0.390	0.788	0.645	0.428	0.760	0.027	0.085	0.125	1.0
InstantCert	0.413	0.772	0.655	0.453	0.733	0.031	0.098	0.135	1.0
RVT-Swarm	0.435	0.760	0.660	0.478	0.764	0.023	0.068	0.113	1.6

VII. LIMITATIONS AND FUTURE WORK

This work should be read as a research prototype with promising algorithmic structure, not as a finished benchmark artifact. The main limitations are:

- **Short-horizon recoverability surrogate.** Recoverability labels are derived from a heuristic expert over a finite horizon rather than from a globally optimal planner or a formally certified reachability computation.
- **Prototype baselines.** The historical baselines are normalized in-house implementations in a common simulator, not exact reproductions of external packages.
- **Single-seed reporting.** The current paper reports one full prototype sweep. A submission intended for a top-tier venue should include multi-seed confidence intervals, statistical testing, and a more mature baseline suite.
- **Simplified shield and dynamics.** The shield uses iterative half-plane projection, and the environment is 2D with simplified kinematics. Extensions to nonholonomic and 3D platforms remain open.

The clearest next steps are to (i) align checkpointing and training losses more directly with rollout success, (ii) strengthen the recoverability label generation with better local planning or longer horizons, and (iii) evaluate the method over multiple seeds and richer baseline reproductions.

VIII. CONCLUSION

We presented RVT-Swarm, a decentralized graph-based controller that combines continuous control, discrete topology adaptation, and a learned recoverability score map for swarm formation control in constrained environments. The current implementation demonstrates that this design can outperform simpler learned baselines on the main benchmark and can reduce deadlock and irreversible collapse, while also revealing through ablation that the recoverability-selector-shield coupling still leaves headroom for improvement.

REFERENCES

- [1] K. Garg, S. Zhang, O. So, C. Dawson, and C. Fan, “Learning safe control for multi-robot systems: Methods, verification, and open challenges,” *Annual Reviews in Control*, vol. 57, p. 100948, 2024.
- [2] Z. Deng, P. Gao, W. J. Jose, and H. Zhang, “Multi-robot collaborative navigation with formation adaptation,” *arXiv preprint arXiv:2404.01618*, 2024.
- [3] Y. Xie, C. Yu, H. Zang, F. Gao, W. Tang, J. Huang, J. Chen, B. Xu, Y. Wu, and Y. Wang, “Multi-UAV formation control with static and dynamic obstacle avoidance via reinforcement learning,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 20 410–20 417.
- [4] B. Muthusamy, N. Mittal, Q. Li, A. Sahu, T. Chan, and A. Prorok, “Generalizability of graph neural networks for decentralized unlabeled motion planning,” *arXiv preprint arXiv:2404.16089*, 2024.
- [5] A. Goeckner, Y. Sui, N. Martinet, X. Li, and Q. Zhu, “Graph neural network-based multi-agent reinforcement learning for resilient distributed coordination of multi-robot systems,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.
- [6] S. Zhang, O. So, K. Garg, and C. Fan, “GCBF+: A neural graph control barrier function framework for distributed safe multi-agent control,” *IEEE Transactions on Robotics*, vol. 41, pp. 1533–1552, 2025.
- [7] B. A. Butler, C. H. Leung, and P. E. Paré, “Collaborative safety-critical formation control with obstacle avoidance,” *arXiv preprint arXiv:2410.03885*, 2024.
- [8] S. Zhang, O. So, M. Black, and C. Fan, “Discrete GCBF proximal policy optimization for multi-agent safe optimal control,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.
- [9] J. van den Berg, S. J. Guy, M. Lin, and D. Manocha, “Reciprocal n -body collision avoidance,” in *Robotics Research*. Springer, 2011, pp. 3–19.
- [10] A. Abate, S. Bogomolov, A. Edwards, K. Potomkin, S. Soudjani, and P. Zuliani, “Safe reach set computation via neural barrier certificates,” *IFAC-PapersOnLine*, vol. 58, no. 11, pp. 107–114, 2024.
- [11] G. Li, H. Mirinezhad, K. Garg, J. Sun, C. Fan, B. Ghahesifard, A. A. Ahmadi, and A. K. Sojoudi, “Certifiable reachability learning using a new lipschitz continuous value function,” *IEEE Robotics and Automation Letters*, vol. 10, no. 4, pp. 3582–3589, 2025.
- [12] Y. Jung, M. Kim, H. J. Lim, and M. Pavone, “RAIL: Reachability-aided imitation learning for safe policy execution,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 4903–4909.

APPENDIX

APPENDIX: CURRENT HYPERPARAMETERS

TABLE IV: Current hyperparameter settings in the released prototype.

Parameter	Value
<i>Environment</i>	
Workspace	12.0 $\times$ 12.0 m
Δt	0.15 s
$v_{\max} / u_{\max}$	0.9 / 0.6
Robot / obstacle radius	0.18 / 0.35
LiDAR rays / range / FOV	36 / 3.0 m / 270 $^\circ$
Team sizes	{4, 8, 12, 16}
Scenarios	open, cluttered, narrow, dynamic
<i>Model</i>	
Node feature dim / edge dim	68 / 11
Hidden dim / message passes	128 / 3
Graph neighborhood k	6
Aux gradient scale	0.2
<i>Training</i>	
Expert episodes	500
Batch size	32
Learning rate / weight decay	$3 \times 10^{-4} / 10^{-5}$
Recoverability horizon H	14
λ_a / λ_r	5.0 / 0.02
λ_τ / λ_x	0.01 / 0.004
RVT warmup epochs	50
Early stopping patience	40
RVT max epochs	300
GNN / InstantCert max epochs	120 / 120
<i>Rollout validation</i>	
Enabled	yes
Interval	every 5 epochs
Scenarios	narrow passage, dynamic obstacles
Team sizes	8, 16
Episodes per setting	2
<i>Inference</i>	
Switch hysteresis	0.08
Topology cooldown	12 steps
Topology temperature	1.5
Shield risk threshold	0.85
Max shield blend	0.06
<i>Evaluation</i>	
Episodes per setting	25